EditorToolbar - 編集ツールバーコンポーネント

1. 概要

1.1 コンポーネントの目的

EditorToolbarは、Petri Netエディタ（PetriNetEditorInner）で使用される編集ツールバーコンポーネントです。ノード追加、Undo/Redo、保存、バリデーション、ワークフロー追加、Draft管理など、編集に必要な全機能をツールバーボタンとして提供します。

主要機能:

- ノード追加（Material、Condition、Result、Transition）
- Undo/Redo操作
- 保存機能
- Document編集
- バリデーション実行
- ワークフロー追加
- Draft管理
- 自動保存トグル
- ビューモード切替（ペトリネット/テンプレート）

1.2 使用場面

- PetriNetEditorInner内で常に表示
- React Flowキャンバスの上部に配置
- エディタの全編集機能への入口として機能

1.3 アクセス権限

- **必須ロール**: Experiment Admin
- **認証**: JWT + Session Hybrid認証

---

2. 画面構成

2.1 レイアウト構造

```
┌──────────────────────────────────────────────────────────────────────────┐
│ [ビュー切替] | [Material] [Condition] [Result] | [Transition] | [ワークフロー] |
│ | [Document] | [バリデーション] | [保存] [Draft] | [Undo] [Redo] |          |
│ ... 右側スペーサー ... | [自動保存トグル] | ヒント表示                        |
└──────────────────────────────────────────────────────────────────────────┘
```

2.2 主要UI要素

2.2.1 ビュー切替（Segmented）

- **位置**: 左端
- **オプション**:
  - ペトリネット（NodeIndexOutlinedアイコン）
  - テンプレート（UnorderedListOutlinedアイコン）

2.2.2 ノード追加ボタン群

- **Material Place**（PlusCircleOutlined）: Material Placeノード追加
- **Condition Place**（BorderOutlined）: Condition Placeノード追加
- **Result Place**（RadiusSettingOutlined）: Result Placeノード追加
- **Transition**（SwapOutlined）: Transitionノード追加

2.2.3 ワークフロー追加ボタン

- **アイコン**: ApartmentOutlined
- **状態**:
  - デフォルト: 通常ボタン
  - アクティブ時: Primaryボタン（青色）

- **ラベル**:
  - デフォルト: "ワークフロー"
  - アクティブ時: "ワークフロー (有効)"

2.2.4 Document編集ボタン

- **アイコン**: FileTextOutlined
- **ラベル**: "Document"

2.2.5 バリデーションボタン

- **アイコン**: CheckCircleOutlined
- **ラベル**:
  - デフォルト: "バリデーション"
  - 実行中: "検証中..."
- **状態**: loading（isValidating）

2.2.6 保存ボタン

- **アイコン**: SaveOutlined
- **タイプ**: primary（青色）
- **ラベル**: "保存"
- **状態**: disabled（canSaveがfalseの場合）
- **ツールチップ**: "保存 (Ctrl+S)"

2.2.7 Draft管理ボタン

- **アイコン**: FolderOpenOutlined
- **ラベル**: "Draft"
- **ツールチップ**: "Draft管理（一時保存・読み込み）"

2.2.8 Undo/Redoボタン

- **Undo**（UndoOutlined）:
  - ツールチップ: "Undo (Ctrl+Z)"
  - disabled: canUndoがfalse
- **Redo**（RedoOutlined）:
  - ツールチップ: "Redo (Ctrl+Y)"
  - disabled: canRedoがfalse

2.2.9 自動保存トグル

- **位置**: 右側（スペーサーの後）
- **要素**:
  - ラベル: "自動保存"
  - Switch（small）
  - 最終保存時刻表示（HH:MM:SS形式）

2.2.10 ヒント表示

- **位置**: 最右端
- **テキスト**: "ヒント: Ctrl+Sで保存 | Ctrl+Z/Yで取り消し/やり直し | M/C/R/T+クリックでノード追加"
- **スタイル**: secondary、11px

---

3. 機能仕様

3.1 Props定義

```
export interface EditorToolbarProps {
  // ノード追加
  onAddNode: (nodeType: 'material' | 'condition' | 'result' | 'transition') => void;
```

```
  // Undo/Redo
  onUndo: () => void;
  onRedo: () => void;
  canUndo: boolean;
  canRedo: boolean;

  // 保存
  onSave: () => void;
  canSave: boolean;

  // Document編集
  onEditDocument: () => void;

  // バリデーション
  onValidate: () => void;
  isValidating: boolean;

  // ワークフロー追加
  onAddWorkflow: () => void;
  workflowModeActive: boolean;

  // Draft管理
  onOpenDraftManager?: () => void;

  // 自動保存
  autoSaveEnabled?: boolean;
  onAutoSaveToggle?: (checked: boolean) => void;
  lastAutoSaveTime?: string | Date | null;

  // ビューモード切替
  viewMode?: 'petriNet' | 'templates';
  onViewModeChange?: (mode: 'petriNet' | 'templates') => void;
}
```

Props説明:

Prop名	型	必須	説明
onAddNode	(nodeType) => void	✅	ノード追加時のコールバック
onUndo	() => void	✅	Undo実行コールバック
onRedo	() => void	✅	Redo実行コールバック
canUndo	boolean	✅	Undo可能かどうか
canRedo	boolean	✅	Redo可能かどうか
onSave	() => void	✅	保存コールバック

canSave	boolean	✅	保存可能かどうか（ノードがない場合はfalse）
onEditDocument	() => void	✅	Document編集ダイアログを開くコールバック
onValidate	() => void	✅	バリデーション実行コールバック
isValidating	boolean	✅	バリデーション実行中かどうか
onAddWorkflow	() => void	✅	ワークフロー追加モード切替コールバック
workflowModeActive	boolean	✅	ワークフローモードがアクティブかどうか
onOpenDraftManager	() => void	-	Draft管理ダイアログを開くコールバック
autoSaveEnabled	boolean	-	自動保存が有効かどうか
onAutoSaveToggle	(checked) => void	-	自動保存トグル変更コールバック
lastAutoSaveTime	string | Date | null	-	最終自動保存時刻
viewMode	'petriNet' | 'templates'	-	現在のビューモード
onViewModeChange	(mode) => void	-	ビューモード変更コールバック

3.2 ノード追加

3.2.1 処理フロー

```
// ボタンクリック時
onClick={() => onAddNode('material')}
onClick={() => onAddNode('condition')}
onClick={() => onAddNode('result')}
onClick={() => onAddNode('transition')}
```

3.2.2 UI動作

- ユーザー操作: ノード追加ボタンをクリック
- 親コンポーネント（PetriNetEditorInner）: handleAddNode実行
- 結果: キャンバスにランダム位置でノードが追加される

3.3 Undo/Redo

3.3.1 処理フロー

```
// Undoボタンクリック時
onClick={onUndo}
disabled={!canUndo}

// Redoボタンクリック時
onClick={onRedo}
disabled={!canRedo}
```

3.3.2 UI動作

- ユーザー操作: Undo/Redoボタンをクリック（またはCtrl+Z/Y）
- 親コンポーネント: useHistoryフックのundo/redo実行
- 結果: ノード・エッジの状態が履歴から復元される
- ボタン状態: 履歴がない場合はdisabled

3.4 保存

3.4.1 処理フロー

```
// 保存ボタンクリック時
onClick={onSave}
disabled={!canSave}
```

3.4.2 UI動作

- ユーザー操作: 保存ボタンをクリック（またはCtrl+S）
- 親コンポーネント: handleOpenSaveConfirmDialog実行
- 結果:
    1. バリデーション実行
    2. SaveConfirmDialogが開く
    3. ユーザーが保存先（S3/ローカル）を選択
    4. MAIML保存実行

- ボタン状態: ノードがない場合はdisabled

3.5 バリデーション

3.5.1 処理フロー

```
// バリデーションボタンクリック時
onClick={onValidate}
loading={isValidating}
```

3.5.2 UI動作

- ユーザー操作: バリデーションボタンをクリック
- 親コンポーネント: handleOpenValidationPanel実行
- 結果:
    1. MAIML生成
    2. バリデーションAPI呼び出し
    3. ValidationPanel表示
    4. エラー・警告の詳細表示
- ボタン状態: 実行中はloading、ラベルが"検証中..."に変化

3.6 ワークフロー追加

3.6.1 処理フロー

```
// ワークフロー追加ボタンクリック時
onClick={onAddWorkflow}
type={workflowModeActive ? 'primary' : 'default'}
```

3.6.2 UI動作

- ユーザー操作: ワークフローボタンをクリック
- 親コンポーネント: handleWorkflowModeToggle実行
- 結果:
  - モード無効→有効: ボタンが青色に変化、ラベルが"ワークフロー (有効)"に変化
  - モード有効→無効: ボタンが通常色に戻る
- 次の操作:
  - ユーザーがキャンバスをクリック
  - WorkflowSelectorDialog表示
  - ファイル選択後、選択位置にワークフロー追加

3.7 Document編集

3.7.1 処理フロー

```
// Document編集ボタンクリック時
onClick={onEditDocument}
```

3.7.2 UI動作

- ユーザー操作: Documentボタンをクリック
- 親コンポーネント: handleOpenDocumentEditor実行
- 結果: DocumentEditDialog表示

3.8 Draft管理

3.8.1 処理フロー

```
// Draft管理ボタンクリック時（オプショナル）
{onOpenDraftManager && (
  <Button
    onClick={onOpenDraftManager}
```

```
  >
    Draft
  </Button>
)}
```

3.8.2 UI動作

- ユーザー操作: Draftボタンをクリック
- 親コンポーネント: handleOpenDraftManager実行
- 結果: DraftManager表示

3.9 自動保存トグル

3.9.1 処理フロー

```
// 自動保存トグル変更時（オプショナル）
{onAutoSaveToggle && (
  <Switch
    checked={autoSaveEnabled || false}
    onChange={onAutoSaveToggle}
  />
)}
```

3.9.2 UI動作

- ユーザー操作: 自動保存スイッチをON/OFF
- 親コンポーネント: setAutoSaveEnabled実行
- 結果: useDraftManagerフックの自動保存機能が有効/無効化
- 最終保存時刻表示: lastAutoSaveTimeがあれば"HH:MM:SS"形式で表示

3.10 ビューモード切替

3.10.1 処理フロー

```
// ビューモード切替時（オプショナル）
{onViewModeChange && viewMode && (
  <Segmented
    value={viewMode}
    onChange={(value) => onViewModeChange(value as 'petriNet' | 'templates')}
    options={[
      { label: 'ペトリネット', value: 'petriNet' },
      { label: 'テンプレート', value: 'templates' },
    ]}
  />
)}
```

3.10.2 UI動作

- ユーザー操作: Segmentedコントロールをクリック
- 親コンポーネント: handleViewModeChange実行
- 結果:
  - ペトリネット: React Flowキャンバス表示
  - テンプレート: TemplateList表示

---

4. ユーティリティ関数

4.1 最終保存時刻のフォーマット

```
const formatAutoSaveTime = (time: string | Date | null | undefined): string => {
  if (!time) return '';
  const date = new Date(time);
  const hours = String(date.getHours()).padStart(2, '0');
  const minutes = String(date.getMinutes()).padStart(2, '0');
  const seconds = String(date.getSeconds()).padStart(2, '0');
  return `${hours}:${minutes}:${seconds}`;
};
```

機能:

- string型のISO形式日時またはDate型を"HH:MM:SS"形式にフォーマット
- nullまたはundefinedの場合は空文字を返す

---

5. スタイル仕様

5.1 コンテナスタイル

```
style={{
  padding: '8px 16px',
  borderBottom: '1px solid #f0f0f0',
  backgroundColor: '#fff',
  flexWrap: 'wrap',
  gap: 8,
}}
```

説明:

- パディング: 上下8px、左右16px
- 下ボーダー: 1px、#f0f0f0
- 背景色: 白
- フレックスラップ: 有効（小画面時に折り返し）
- ギャップ: 8px

5.2 Dividerスタイル

```
<Divider type="vertical" style={{ height: 24 }} />
```

説明:

- タイプ: 縦線
- 高さ: 24px

5.3 テキストスタイル

ヒント表示

```
style={{ fontSize: 11, marginLeft: 16 }}
type="secondary"
```

自動保存時刻

```
style={{ fontSize: 11 }}
type="secondary"
```

---

6. ボタン状態管理

6.1 Undoボタン

- **有効**: canUndo === true
- **無効**: canUndo === false
- **ツールチップ**: 常に表示

6.2 Redoボタン

- **有効**: canRedo === true
- **無効**: canRedo === false
- **ツールチップ**: 常に表示

6.3 保存ボタン

- **有効**: canSave === true（ノードが1つ以上）
- **無効**: canSave === false（ノードが0個）

- **タイプ**: primary（青色）

6.4 バリデーションボタン

- **通常**: loading === false、ラベル"バリデーション"
- **実行中**: loading === true、ラベル"検証中..."

6.5 ワークフローボタン

- **通常**: type="default"、ラベル"ワークフロー"
- **アクティブ**: type="primary"、ラベル"ワークフロー (有効)"

---

7. エラーハンドリング

7.1 エラーケース

このコンポーネントは純粋なプレゼンテーションコンポーネントのため、エラーハンドリングは親コンポーネント（PetriNetEditorInner）で実施されます。

7.2 Props検証

TypeScriptの型チェックにより、Propsの型安全性が保証されています。

---

8. 実装ファイル

8.1 ファイルパス

- **メインファイル**: `frontend/src/components/editorv2/components/EditorToolbar.tsx`

8.2 依存コンポーネント

- Ant Design:
  - Space
  - Button
  - Divider
  - Tooltip
  - Switch
  - Typography
  - Segmented
  - Flex
- Ant Design Icons:
  - SaveOutlined
  - UndoOutlined
  - RedoOutlined
  - FileTextOutlined
  - CheckCircleOutlined
  - ApartmentOutlined
  - FolderOpenOutlined
  - NodeIndexOutlined
  - UnorderedListOutlined
  - PlusCircleOutlined
  - BorderOutlined
  - RadiusSettingOutlined
  - SwapOutlined

8.3 親コンポーネント

- PetriNetEditorInner

---

9. 変更履歴

日付	変更内容	担当者
2026-03-12	実装ベースで仕様書を新規作成	Cline
2026-01-29	Material-UI → Ant Design変換	Development Team